```
$num = -9223372036854775745;

print decbin($num) . PHP_EOL;
print decbin($num << 1) . PHP_EOL;
print decbin($num >> 1) . PHP_EOL;
var_dump($num << 1);
var_dump($num >> 1);

Output:
1000000000000000000000000000000000000000000000000000000000111111
1111110
1100000000000000000000000000000000000000000000000000000011111
int(126)
int(-4611686018427387873)
```

At first glance you might not fully understand what's happening here. Bit shifting to the left turned our huge negative integer into a much smaller positive one! If left-shift multiplies by two, what happened?

The answer lies in the leftmost bit. Notice when we left shift, the leftmost *1* is set to the value on the left, which is *0*. This transforms the integer into a positive value, and also greatly affects the magnitude. As for the change in magnitude (from 10^{18} to 10^{2}), once you've read the section on limits and precision later on, you'll understand.

There are also logical bitwise operators - AND &, OR |, and XOR ^ - that can also be used for manipulation:

```
$leastSignificant = bindec('00011111');
$mostSignificant = bindec('11111000');

var_dump(decbin($mostSignificant & $leastSignificant));
var_dump(decbin($mostSignificant | $leastSignificant));
var_dump(decbin($mostSignificant ^ $leastSignificant));

Output:
string(5) "11000"
string(8) "11111111"
string(8) "11100111"
```